VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

MATHEMATICAL MODELING (CO2011)

Large assignment

"2D Cutting stock problem"

Instructor(s): M.Sc Trần Hồng Tài

Students: Nguyễn Văn An - 2352013 *(Group CC04 - Team 02, **Leader**)*
Đào Duy Đạt - 2352223*(Group CC04 - Team 02)*
Quách Hoàng Phú - 2352922 *(Group CC04 - Team 02)*
Lê Đặng Khánh Quỳnh - 2353037 *(Group CC04 - Team 02)*
Lê Hữu Triều - 2353219 *(Group CC04 - Team 02)*

HO CHI MINH CITY, DECEMBER 2024

Contents

Member list & Workload **2**

1 Introduction **3**
1.1 A brief history . . . 3

2 Background **3**
2.1 Integer Linear Programming formulation of the problem . . . 3
2.2 Classification and variations . . . 5
2.2.1 Important notes . . . 5
2.2.2 Basic characteristics and their descriptions . . . 5
2.2.3 Knapsack problem and strip cutting/packing problem . . . 6
2.2.4 An attempt to classify the problem . . . 7
2.3 The exact and approximation approaches . . . 7
2.3.1 Exact method . . . 7
2.3.2 Heuristic methods for approximations . . . 8
2.3.2.1 Greedy Algorithms . . . 8
2.3.2.2 A heuristic based on Ortmann's algorithm of Multiple bin packing problem 8
2.3.3 Advantages and disadvantages . . . 10
2.4 Case study on the cutting stock problem . . . 10
2.4.1 Cutting stock problem in hardboard industry . . . 10
2.4.2 Cutting stock problem in stone manufacturing . . . 12

3 Algorithm Analysis **13**
3.1 Genetic algorithm . . . 13
3.1.1 A quick recap about this algorithm . . . 13
3.1.2 Experimental results and algorithm analysis . . . 16
3.1.3 Advantages and disadvantages . . . 17
3.2 Greedy algorithm . . . 17
3.2.1 Pseudocode explanation . . . 17
3.2.2 Trim loss and Filled ratio analysis . . . 18
3.2.3 Advantages, disadvantages and further discussion . . . 19

4 Conclusion **19**

References **21**

Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Nguyễn Văn An	2352013	Research Genetics Algorithm and perform analysis	100%
2	Đào Duy Đạt	2352223	LaTeX and problem formulation	100%
3	Quách Hoàng Phú	2352922	LaTeX and problem formulation	100%
4	Lê Đặng Khánh Quỳnh	2353037	Research case study, proofread and variations of the problem	100%
5	Lê Hữu Triều	2353219	Research Greedy Algorithm and code optimizing	100%

Table 1: Member list & workload

1 Introduction

The Cutting Stock Problem (CSP) is a classic combinatorial optimization problem in operations research. It arises in industries where raw materials (like metal sheets, wood planks, paper rolls, or fabrics) are cut into smaller pieces to fulfill specific size demands. The objective is typically to minimize waste or cost while meeting the required demands.
Our goal in this assignment is to successfully model the Cutting Stock Problem into an Integer Linear Programming (ILP) problem as well as providing some algorithms to solve it.

1.1 A brief history

Cutting problems, in general, have probably emerged in many industries such as textile and paper manufacturing for a long time. However, not until 1939 was the cutting stock problem first formally formulated in 1939 by Kantovorich [9] who considered the minimization of scrap – a one-dimensional (1D) cutting stock problem in which a number of short pieces of material are to be cut from a limited number of longer items in order to minimize watse. Kantovorich also introduced the concept of linear programming to model this problem mathematically. Since then, the interest in CSP as well as the linear programming model has grown, one of the most noticeable published work are Eisemann's *The trim problem* (1957) [7], Gilmore and Gomory's *A linear programming approach to the cutting-stock problem* (1961) [11], Hinxman's *The trim-loss and assortment problems* (1980)[1], etc. In the latter half of the 20th century, according to Y. Lirov [17], a survey of the literature on CSP reveals that the scope of this famous problem applications has expanded from pallet loading, packing, and industrial production planning to computer operations to telecommunications. Because of the cutting problem's important role in many applications and its high-demanding computational cost for exact methods, a large number of heuristics are discussed in order to meet various specific demands in different industries.

2 Background

In this section, we aim to offer an overall inspection into the two-dimensional cutting stock problem as well as some related frameworks (i.e. knapsack problem and strip packing problem) which facilitate the main problem (i.e. cutting stock problem)'s understanding. All is for the main purpose of having a firm grasp on the fundamental aspects and different ways of visualization of the cutting problem. Regarding a thorough analysis and implementation of the 2D cutting stock problem in this assignment, section 3 will cover this.

2.1 Integer Linear Programming formulation of the problem

First, we begin by defining the problem mathematically:

- **Inputs:**
  - A set large stock sheets of various dimensions $L_j \times W_j$.
  - A list of required piece sizes $l_i \times w_i$ and their corresponding quantities (demands) d_i.
- **Output**
  - Arrangement of smaller pieces on each large sheet to ensure total demand is satisfied.
  - Minimize the stock area to optimize sheet usage.

Next, we need to define some decision variables used for this problem.

- $x_{n,k}$: binary variable that indicates whether cutting pattern k is applied to stock n.

$$x_{n,k} \in \{0, 1\} \quad \forall n, k$$

- $c_{n,k}$: A column vector that represents the number of sheets of each type cut from pattern k on the n_{th} stock.
- w_n: The waste area of the n_{th} stock after the cutting patterns.

- If $x_{n,k} = 0$, then $w_n = 0$ because this stock is not used.
- If $x_{n,k} = 1$, then
$$w_n = L_n \times W_n - \sum_j l_j \times w_j \times c_{n,j}$$
where j refers to the j-th type in the list of demands.

The next step is to focus on the objective. Since the goal of this problem is to minimize waste, the objective function should aim to minimize the sum of all remaining areas w_n. Therefore, our objective function would be
$$\text{Minimize: } S = \sum_n w_n$$
Now that we have defined the objective function, the next step is to define the constraints of the problem. These constraints ensure that the solution is feasible and satisfies all the requirements of the cutting process

- **Stock Usage Constraint:**
The decision variable $x_{n,k}$ is binary, meaning it can either be 0 or 1:
$$x_{n,k} \in \{0, 1\} \quad \forall n, k$$
This constraint ensures that each cutting pattern is either selected or not, as per the decision variable's definition.

- **Demand Satisfaction Constraint:**
To ensure that all customer demands are fully satisfied, the sum of the number of pieces cut from each stock must equal the demand for each piece type. This is because, to achieve the most optimized solution , we will use the equality sign =, ensuring that the total number of pieces exactly matches the required quantities. Mathematically, we express this constraint as:
$$\sum_n \sum_k c_{n,k} \cdot x_{n,k} = d_i, \forall i$$
Where:
  - $c_{n,k}$ is the number of pieces of type i that can be cut from the pattern k on stock n.
  - $x_{n,k}$ is the binary decision variable indicating whether the cutting pattern k is applied on stock n.
  - d_i is the demand for the i-th piece type.

- **Remaining Area Constraint:**
The remaining area on each stock must be greater than or equal to 0, ensuring no negative remaining area after cutting:
$$w_n \geq 0 \quad \forall n$$
where w_n is the remaining area on stock n.

Relaxations: A solution for real-world applications

In practical scenarios, the strict equality constraint = often used in CSP formulations to minimize waste or maximize material utilization may not always be feasible or necessary. Instead, relaxing this constraint to allow inequalities $\geq$ provides greater flexibility and adaptability, especially in real-world applications. This section discusses the reasoning behind such relaxations, their implications, and their relevance to broader problem domains.

Such relaxation might be beneficial in practice for the following reasons:

- In an ideal optimization setting, enforcing = ensures that the exact demand is met without producing excess stock. This makes the problem substantially more difficult to solve. Furthermore, such strictness may limits feasible solutions or even results in suboptimal use of raw material. That is, sometimes cutting patterns that slightly exceed demand can often achieve lower material waste compared to those that strictly meet it.

- By allowing $\geq$, the model permits producing more than the exact demand when it leads to better utilization of materials or minimizes waste. For example, depends on the context, one can cut the materials in quantities that exceed the demand so they may use them for later needs, or store them for the next demands, etc.

2.2 Classification and variations

The cutting stock problem, and cutting problems in general, have various variations attributed to different applications in many industries, not to mention the real-life constraints in the cutting equipment or the cutting process. Therefore, this section attempts to clarify some basic characteristics to differentiate many variations of the cutting stock problems, with the inspiration from the typology of Dyschoff [5] and Wascher et. al. [3], along with a reference to Ortmann's dissertation [2]. Note, this is not a complete typology.

2.2.1 Important notes

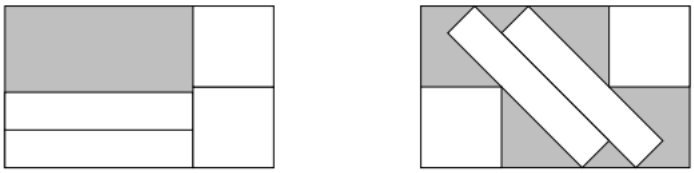
First, some frequently used terms when discussing cutting and packing problems will be introduced. "Large objects" or "an object" referred generally to the stock material in cutting stock problems or the bins in bin packing problems. Meanwhile, "small items" or "an item" referred to the requested stock to be cut in smaller sizes or the items to be packed into the bins. Also, the scope of this section only covers the problems where all of the list of items and objects have been known initially.

Second, the close relationship between cutting and packing problem is emphasized. More specifically, the latter can be seen as "packing" small items into the space occupied by large objects and the former may be considered as "cutting" the empty space inside the bins where the remaining empty space is "trim loss". Because of this strong relation between cutting and packing problem, this section can be applied to both of these two classes of problems and in many cases, a cutting problem can be equivalently considered as a packing problem and vice versa.

2.2.2 Basic characteristics and their descriptions

- **Dimensionality:**
  - One dimension such as when dealing with rolls or bars, or two dimensions such as when dealing with metal sheets or wooden panels, or three dimensions, etc. At higher dimensions, the search space and the possible combination increases enormously.
  - All of the dimensions of the objects are fixed, or one or more dimensions are variables. For example, strip packing problems is the case when one dimension e.g. height is a variable.
- **The kind of assignment:**
  - All of the large objects should be assigned to a selection of small items.
  - All of the small items should be assigned to a selection of large objects. This is commonly seen in cutting and packing problems.
- **The number of large objects**:
  - One large objects such as single stock cutting problems and single bin packing problems.
  - Multiple large objects such as multiple stock cutting problems and multiple bin packing problems.
- **Assortment of large objects and small items**: When there are multiple items/objects, the assortment of them is concerned. Although we do not sure about the definite criteria, a recommendation for it is the ratio between the number of identical items/objects and the number of unique items/objects.
  - Completely identical: All of the items/objects are completely identical in size, length, etc.
  - Weakly heterogeneous: Many of the items/objects are identical. An example extreme case is when there are 4 unique items with each of the 4 items' demand is 1000, 5000, 3000, 7000, respectively. It can be seen that the number of identical items (for each item) are relatively greater than the number of unique items.
  - Strongly heterogeneous: Very few of the items/objects are identical.
- **Shape and rotation:** Many of the cutting or packing problems are concerned with rectangular, but there are also non-rectangular cases such as disks, and irregular shapes such as in the clothing industry. Furthermore, rotation refers to whether the items are rotated or not.

- **The type of cutting/packing:** Some common type of cutting or packing will be introduced. An important point to highlight is that a problem may fall into a combination of these common type of cutting, or may not.
  - Orthogonality: As illustrate in Figure 1, an orthogonal packing/cutting means that the edges of the small items must be parallel or perpendicular to the edges of the large objects into which cutting/packing occurs.



(a) Orthogonal Packing (b) Non-Orthogonal Packing

Figure 1: A comparison between orthogonal packing and non-orthogonal packing

  - Guillotine cut: All cuts must go from one edge of the rectangle to be cut to the opposite edge [10]
- **Objective:** This may not be considered as a characteristics, but it is important to differentiate each problem or determine how much each problem is related to each other. The following is the common objectives appearing in the cutting and packing problem:
  - Minimize waste: When cutting stock, there is trim loss and when packing bin, there is the empty space left in the bin after packing items. In some industries, minimizing the waste can mean a reduction in cost. In bin packing, minimizing the waste can be equivalent to maximizing the consumed area of the items in the bins.
  - Minimize the number of stock/bin: This can seen as minimize the number of patterns if each stock/bin is assigned with a pattern. In many applications, each bin/stock can cost a certain amount and thus, minimizing the number consumed can also lead to the reduction in production cost.
  - Maximize the profit: Some variations of the cutting stock problems assign a value or profit for each small items, with the limitation on the number of selected small items. This version of the problem appears in the problem of cutting steel, glass plates or cutting wood plates to make furniture [10].

2.2.3 Knapsack problem and strip cutting/packing problem

Knapsack problem and strip cutting problem can be seen as special cases of cutting stock problem. Many cutting stock problem algorithms are built on the basis of many of these special cases and thus, an brief explanation to why these cases are related to the cutting stock problem will be presented.

First, the knapsack problem also fall within the scope of combinatorial optimization. The problem involves a set of items, each assigned a weight and value, and a knapsack with a certain capacity. The objective is to choose a set of items packed into the knapsack so that the total value is maximized. In comparison with the cutting stock problem, an example of the equivalence, based on the article of Christofides's and Whitlock's [10], can be shown that:

- The knapsack is the stock sheet, with the sheet's area is the capacity.
- There is also a constraint that restricts the maximum number of items of each type to be cut. This is similar to the number of items of each type available to be packed into the knapsack.
- The weight of each item is the consumed area of each item
- A value can be assigned to each item in some variation of the cutting stock problem, and this works the same as the value in the knapsack problem.
- The objective is to maximize the value.

In this example of the cutting stock problem, a noteworthy piece of information is that there is a single large object, and it must be assigned to a selection of small items.

In column generation method (in section 2.3.1), after formulate the master problem acting as the primal problem, the dual problem's decision variables are used to indicate how profitable it is to add the associate item to a new pattern. Then, to find the best pattern to add, we solve the knapsack problem (known as subproblem in this context) with the profit information provided by the dual problem, the weight is the consumed area of each item, the capacity is the area of the stock sheet. [4] [12]

Second, two dimensional strip problems refers to those with a single large object whose one dimension is fixed and the other dimension is unlimited. For the simplicity to approach the cutting stock problem, some heuristics view this problem as a combination of strip cutting problems. Similarly, a multiple bin packing problem can be views as a combination of strip packing problems. In other words, an idea for heuristics is that a multiple large objects problem can be derived from a single large object problem. Hence, explanations for the strip cutting problem (i.e. an example for a single large object problem) can potentially pave the way for understanding some of the heuristics for cutting stock problem. Ortmann's heuristic for multiple bin packing problem is an example for this idea, as presented in section 2.3.2.2.

A more detailed example of the utilization of knapsack and strip cutting problem will be shown in section 2.3.2.2.

2.2.4 An attempt to classify the problem

Based on the listed characteristics, the cutting stock problem discussed in this assignment is classified as follows:

- Two dimension
- All of the small items should be assigned to a selection of large objects.
- Multiple cutting stock problem.
- The assortment of stock material is strongly heterogeneous and each stock is assigned only one pattern. This can be equivalent to each "bin" can only have one "packing pattern".
- The assortment of small items is relatively heterogeneous.
- Only rectangular objects and items.
- The items can be rotated i.e. an item with size $l \times w$ is the same as the one with $w \times l$.
- Feasible cutting patterns are generated by orthogonal cut, and guillotine cut algorithm can also apply for this problem.
- The objective is minimizing waste, as well as minimize the number of used stock if possible. Because each stock is assigned only one pattern, minimizing the stock usage also means minimizing the number of "chosen" (i.e. $x_{n,k} = 1$) patterns.

2.3 The exact and approximation approaches

This section provides a short summary about the exact and approximation approaches to the CSP, describing one of the most common methods for each approach.

2.3.1 Exact method

Exact methods provide an optimal solution by exhaustively exploring the solution space. They are ideal for small to medium-sized problems. Integer Linear Programming (ILP) is commonly attributed to this kind of approach.

Integer Linear Programming (ILP)

As described in Section 2.1, Integer Linear Programming is a type of optimization problem where the variables are integer values and the objective function and equations are linear. For the CSP, we have to find the objective function for minimizing the number of stocks. This section will introduce the Column generation technique, which is used to reduce the search space of the original ILP.

Column generation Column generation is a technique for solving (mixed) integer programming problems in general. This technique was first applied to large real life cutting stock problem by Gilmore and Gomory [11]. The technique can be described as follow:

Consider the standard form of LP:

$$\min c^T x$$

$$\text{s.t.} Ax = b$$

$$x \geq 0$$

Where A is a $m \times n$ matrix with linearly independent rows.
Suppose that n is really large and it is impossible to generate and store the whole matrix. Column generation improves ILP by generating cutting patterns dynamically:

1. **Master Problem:** Solve an ILP with a limited set of initial patterns.
2. **Subproblem:** Use a knapsack problem solver to find new patterns that minimize waste or meet uncovered demand.
3. **Iteration:** Add new patterns to the master problem and repeat until no further improvement is possible.

2.3.2 Heuristic methods for approximations

The exact methods are impractical most of the time due to the sheer amount of calculations one has to perform. Since the problem is in the class NP, there are no ways to solve it fast (i.e. in Polynomial time) unless P = NP, which is widely believed to be false. Heuristics on the other hand, provide near-optimal solutions while still run in a reasonable amount of time, thus make them efficient for large-scale problems. "Near optimal" solutions mean those good enough for certain real-life scenarios, and thus, heuristics provide flexibility and pragmatism.

2.3.2.1 Greedy Algorithms Greedy algorithms are commonly attributed to this kind of approach, which is basically choosing the best solution at every step. A simple (and somewhat naive) greedy algorithm can be described as follow:

- Start with the largest demand or size.
- Assign pieces to raw material until no more pieces fit.
- Repeat until all demands are met.

Greedy is a simple and fast heuristic, but often provides worse result than other kinds of heuristics due to their locally optimized and naive nature and therefore easily gets trapped in a local optimum. Note that, beside from the above approach, there are many other ways to greedily (and cleverly) assign demands for each stock, which gives better results. One of them will be discussed in Section 3 with details.

2.3.2.2 A heuristic based on Ortmann's algorithm of Multiple bin packing problem As stated before, many cutting stock problems share characteristics with bin packing problems, especially the problem in this assignment. Therefore, Ortmann's heuristic [2] of two-staged multiple bin packing problem may be suitable for a cutting stock problem.

a. Overview
First, the stock materials are sorted by decreasing area, with equalities resolved by sorting by increasing perimeter. There are two stages as follows:

- The first stage: Start with the largest stock area, choose a pattern for this stock material by strip cutting algorithm (equivalent to strip packing algorithm of Ortmann's). After the first stage, we will have a list of chosen patterns.
- The second stage: Try to "re-pattern" the stock. This stage is an attempt to "choose again" i.e. to choose a pattern with the same number of items as the chosen patterns but will generate less trim loss.

- First, select the stock with the lowest area consumption pattern i.e. the total area that is consumed in the pattern, this also means selecting the chosen pattern in the first stage with the maximum trim loss (Note that a pattern corresponding to a stock). This stock will be called "the selected stock".
- Search for the "uncut" stock whose area is the smallest and greater than or equal to the area consumption of the pattern in "the selected stock". This means searching for the smallest stock which can be potentially generate a better (i.e. less trim loss) pattern with the same number of items.
- Perform strip cutting the items from "the selected stock" into the smallest uncut stock which we just search for. If we cannot successfully "re-pattern", then we move to the next "selected stock".
- Continue for the next "selected stock" for all of the chosen pattern in the first stage.

b. About the strip cutting/packing problem

Because the equivalence between cutting and packing problems, from now on, we focus on the "the strip packing problem" as presented by Ortmann.

In this section, we take an example of a fixed-width and unlimited-length strip, in which width is the horizontal line of a rectangular and length (also called height) is the vertical one. The objective can be cutting/packing the strip for the minimum length.

Ortmann proposed some algorithms for strip packing problems, among which the knapsack algorithm for level strip packing heuristic will be presented. Level strip packing heuristics partition the strip by means of horizontal lines, thereby creating levels. A new level is initialized above the topmost level if items no longer fit on lower levels. To see how to "cut" with these packing algorithm, Ortmann points out that the level strip packing algorithms yield a two-stage cutting pattern. This means that when the items are cut, the pattern will have to be turned by ninety degrees no more than twice in order for the items to be separated by means of a hypothetical set of linear blades. The first cut would be along the lines representing the levels and the second set of cuts would be to separate the items in each level from each other. This method of describing the solution pattern does not include the turn required for the trimming of waste from the items.

The common ideas in level strip packing heuristics of Ortmann's will be shown:

- First, all items are sorted by decreasing height.
- Select the highest item to initialize the first level.
- Choose a set of items to fit into the level by some criteria, until no more items can be fitted.
- We continue initialize a level by the highest unpacked item, "fully" (i.e. until no more items can be fitted) pack that level and then repeat this process until all of the items are packed.

To have a deeper insight into what "level" means, in Figure 2, A_1 and A_2 is the initializing item of the first level and the second level, respectively. The height of each level is that of its initializing item i.e. the first level = height of A_1, the second level = height of A_2. It is also illustrated that the width of the strip if fixed, and the length of the strip continues upward.

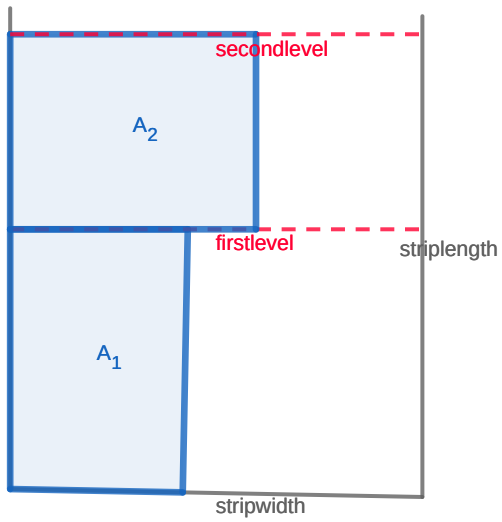


Figure 2: Illustration for level strip packing

For each level with an initializing item, the problem of choosing a set of items to fit into the level can be formulated as a 0-1 knapsack problem, where:

- The profit is the area of the item. To maximize profits means to maximize the consumed areas, which also leads to minimize the wasted space left.
- The weight is the width of the item. Because we initialize a level by the highest unpacked item, all of the remaining unpacked items are guaranteed to have their heights no more taller than the level
- The capacity of the "knapsack" is the width of the strip minus the width of the initializing item.
- Each item is assigned a decision variable which can be zero or one. One means that the item is to be packed in to the level, and zero means the opposite.

2.3.3 Advantages and disadvantages

Each kind of approach has its own pros and cons. For that, we have studied said pros and cons and summarized them in Table 2:

Criteria	**Exact Method**	**Heuristic Method**
Optimality	Guarantees the global optimal solution.	Provides near-optimal solutions.
Speed	Slow, especially for large-scale problems.	Faster and effective for large or complex problems.
Computational Resources	High demands for time and memory.	Lower demands, suitable for systems with limited resources.
Scalability	Difficult to scale for large problems.	Suitable for large-scale and complex problems.
Feasibility	Always guarantees valid solutions that meet all constraints.	May produce invalid solutions if not carefully controlled.
Implementation	Typically more complex.	Simpler and easier to implement in practical scenarios.

Table 2: Exact Method vs Heuristic Method

2.4 Case study on the cutting stock problem

2.4.1 Cutting stock problem in hardboard industry

This case study of Morabito and Garcia [13] considers the hardboard cutting problem in a large Brazilian hardboard industry.

a. Background

Hardboards are plates composed of wood fibers pressed together after many processes. The plates without any kind of defects are cut into smaller ordered items by an automated machine called samba saw. The samba saw is the most important equipment in the cutting process, and most of the constraints in this cutting problem are decided based on the limitation of the samba saw.

There are two stages in the cutting process of the samba saw:

- In the first stage, the longitudinal saws make cuts parallel to the plate length, yielding strips of the same length of the plate
- In the second stage, for each strip in the first stage, the saw makes a parallel cut to the plate width

The current waste material in the cutting process involves approximately 20 tons per day, or 1 million US dollars a year. Such waste consists of good-quality hardboard scraps which are useless for practical purposes because of their small sizes. Part of this waste can be avoided by a more effective production planning of the cutting machine, which does not imply additional investments in capacity.

b. Constraints

In this problem, we assume that the length is the larger dimension and the width is the smaller one.

The samba saw composed of a set of circular saws which produce longitudinal and transversal cuts on the plates. *These two saws' thickness is 5 mm.* This constraints can be addressed by simply add the saw thickness to the dimensions of the items and the hardboard plates.

The saws are able to produce only *orthogonal guillotine cut.* Definition according to the author: an orthogonal guillotine cut is a cut parallel to the plate border that, when applied to a rectangle, produces two rectangles.

The possible cutting pattern produced by the samba saw is said to be *exact two-dimensional two-staged guillotine cutting pattern* (see Figure 3.(a)). The pattern is said to be exact because all items on each strip have the same width i.e. the samba saw is not able to trim these items as in the pattern of Figure 3.(b)

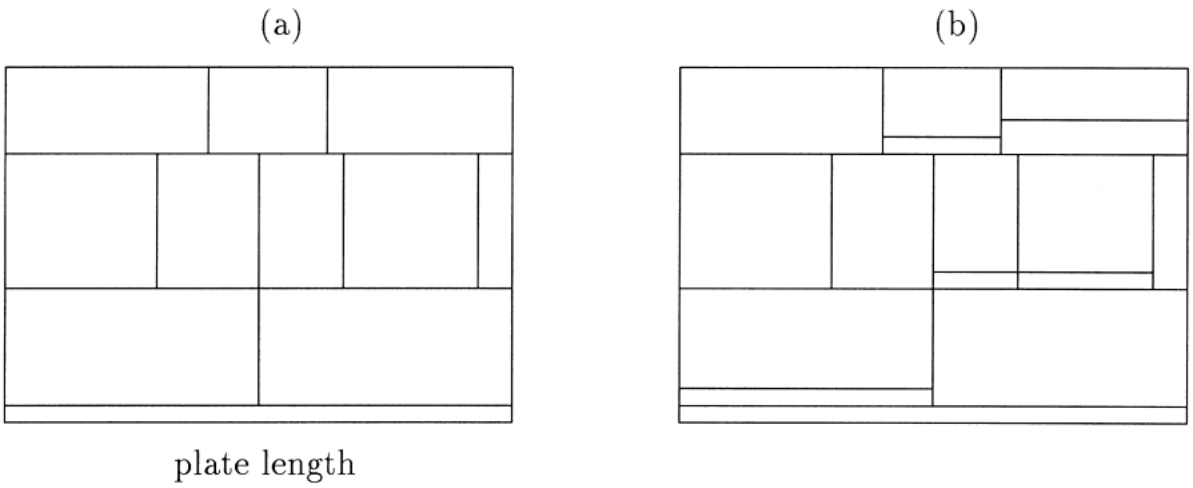


Figure 3: Two-dimensional two-staged guillotine cutting pattern: a) exact, b) not exact

In the first cutting stage, *at most 3 strips* can be yielded

In the second cutting stage, *at most 3 different item lengths* can be yielded. If a strip contains exactly 3 different item lengths, the distance between the largest and smallest item length in the strip is at least 300 mm.

There are, at most, five different item sizes for a cutting pattern, because the samba saw is only able to sort up to five items by size (length and width).

c. Input, output and objective

In this section, we present what the initial information available for the problem (input), what we would expect after solving the cutting problem (output), and at which aim we are striving for (objective).

Input:

- A set of item types of the order list. Each type of item has an ordered quantity and size with length and width.
- The size of the plates of hardboard

Output:

- The number of plates to satisfy the demand
- The cutting patterns optimized for the objective, and the number of times each pattern is cut by the samba saw
- The cutting pattern sequence for each cutting pattern to be cut by the samba saw

Objective:

- Minimize the overall trim loss (or the waste)

d. Remark
According to the author, and from what we have inspected, this problem can be classified as:

- Two dimension
- All items must be produced by a selection of the plates
- Multiple large objects (the hardboard plates)
- All of the objects are identical
- The assortment of the ordered items is weakly heterogeneous i.e. the ordered quantity may be large but the unique ordered type of items is a relatively small number.
- Only rectangulars, and not rotated

We can see that this problem is similar to the column generation model, and the differences between this problem and the problem in this report is the assortment of the large objects and the lack of rotation.

e. The author's approach

The approach proposed here is based on a two-phase modeling of the problem, each one as an integer program, and the use of dynamic programming and implicit enumeration techniques to solve it. Two alternative methods are proposed to solve it:

- Dynamic programming recursive formulas
- Tree-search procedure using a depth-first strategy

2.4.2 Cutting stock problem in stone manufacturing

This case study follows U. Ayasandır's thesis [16]

a. Background and constraints

The main raw materials (marble, granite or ceramic) enter to the factory from their natural sources in the forms of blocks and then after refining and cutting these 3D blocks, we get a number of 2D objects.

These objects, called panels, are of limited availability and have to be cut into a number of small items that customers ask for. Each panel is specified by a cost value depending on the panel's quality level. A large area panel may have lower cost than a smaller one if the former has lower quality level.

The total cut of items must meet exactly as demand, it is not allowed to be exceed the demand. Each item also has a value depending on the area, quality, and prestige of the customer.

b. Input, output and objective
Input:

- A set of different panels, each with limited availability.
- A set of item and its demanding quantity.

Output:

- The number of the panels to be cut
- The cutting pattern of each panel

Objective:

- Maximize the total profit i.e. the total value brought by meeting the demand minus the total cost because of the panel usage. This can also means minimize the panel usage and maximize the total value of the items.

c. Remark and the author's approach
The problem in this case study is rather similar to the problem in this assignment, with exception of the lack of rotation. Although our problem does not assign a cost for each pattern, minimizing the number of pattern is already considered.

The author proposes some heuristics to address the cutting problem where the problem is decomposed into small sub-problems. The heuristics also aim at the idea of prioritizing the cheap panels (i.e. which do not cost much), with larger area preferably.

3 Algorithm Analysis

This section summarizes our research on two different ways to solve the CSP problem, particularly the variation of the CSP with multiple stock sizes and multiple demands.

3.1 Genetic algorithm

3.1.1 A quick recap about this algorithm

A genetic algorithm, in its essence, is the simulation of a population and the changes in that population throughout many generations to find the best individual that serves as the solution. It is a metaheuristic, defined as 'a general heuristic method that is used to guide an underlying local search algorithm to promising regions of the search space containing high-quality solutions' [14]. The solutions of metaheuristics are most of the times just approximations.
The following genetic algorithm is a combination of the works of several papers ([6] [8] [15]) with some modifications of our own to meet the requirements of this assignment. The algorithm is described as follow:

A. **Encoding**
We assign each stock with a number starting from 0, and we do the same for the products. For example, if we have 2 stocks and 8 demands of 4 types then we can encode them as follows:

Stock	Dimension	Encode
S_0	20 x 35	0
S_1	42 x 69	1

Table 3: Stock table

and:

Demand	**Dimension**	**Quantities**	**Encode**
P_0	2 x 3	2	0, 1
P_1	4 x 9	3	2, 3, 4
P_2	5 x 5	2	5, 6
P_3	2 x 5	1	7

Table 4: Demand table

B. **Initialization**

Suppose we have stock encodings and demand encodings, then a chromosome will be an array with length where the first entries are the stock encodings and the remaining will be the demand encodings. Take the above example, two possible chromosomes are:

1	**0**	2	3	1	5	6	0	7	4

0	**1**	3	1	4	5	2	6	7	0

where the bold numbers are the stock encodings and the rest are the demand encodings.
The chromosomes will inhabit the population, and the number of chromosomes will be a fixed number (in our code, it's 20).

C. **Difference and Elimination process**

This is one of the most crucial part of the algorithm because it is used to fit the demands into the stocks. We will use the *Difference and Elimination process* described in Section 4 of Lai and Chan's paper [8]. To put it simply, the *Difference* process can be described as follow:

Suppose we have a rectangle R_1 with a bottom left coordinate of $(x3, y3)$ and an upper right coordinate of $(x4, y4)$, and another rectangle R_2 with a bottom left coordinate of $(x1, y1)$ and an upper right coordinate of $(x2, y2)$. We assume that $x1 \leq x3 \leq x4 \leq x2$ and $y1 \leq y3 \leq y4 \leq y2$, then we define the *Difference* of the rectangle R_1 and R_2 as:

$$\begin{aligned} R_2 - R_1 &= [(x1, y1), (x2, y2)] - [(x3, y3), (x4, y4)] \\ &= [(x1, y1), (x3, y2)], && (1) \\ &\quad [(x4, y1), (x2, y2)], && (2) \\ &\quad [(x1, y1), (x2, y3)], && (3) \\ &\quad [(x1, y4), (x2, y2)]. && (4) \end{aligned}$$

So a difference of two rectangles produces a set of 4 new rectangles (see Figure 4). However, the side lengths of these new rectangles might be zero. The *Elimination* process will handle such cases, where it simply removes the rectangles having any side length equals zero and the rectangles which lie inside another bigger rectangle.

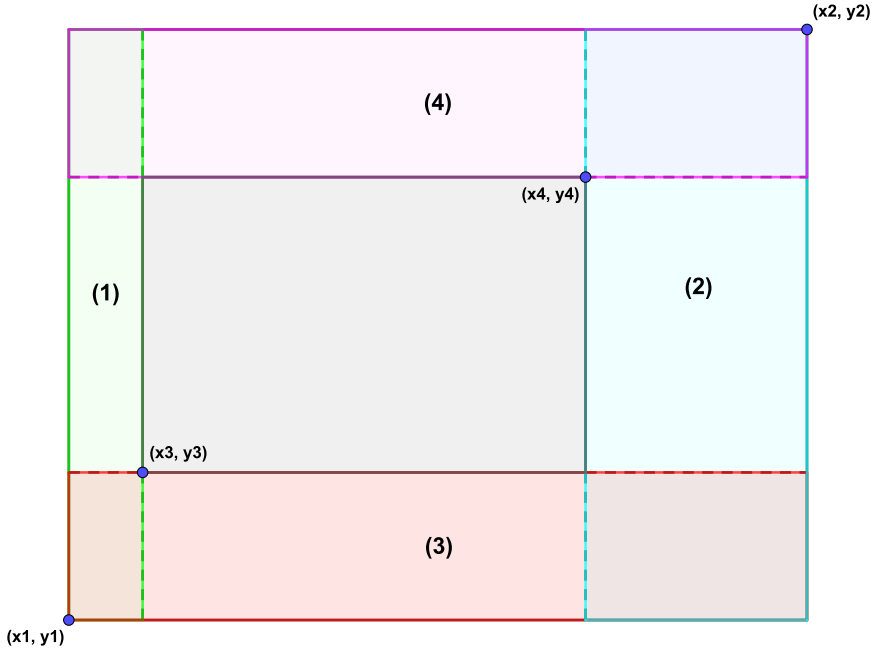


Figure 4: The difference between two rectangles R_2 (the big rectangle) and R_1 (the gray rectangle)

D. **Heuristic placement method**

This process will use the Guillotine cut process above to cut the demands from the stocks, given the chromosome. The final result will determine the quality of the chromosome (i.e. its **score**), which will be discussed in the next section. For now, we focus on implementing the method, which follows the pseudocode in Algorithm 1:

```
Input  : S_i: Sheets to be cut, NS: Number of sheets, NP: Number of pieces
for i ← 1 to NS do
    Let P_j be the piece to cut from S_i;
    for j ← 1 to NP do
        if P_j is not placed on any sheet and number of ERSlists > 0 then
            Let ERS_k be the available ERS (Empty Rectangular Space) in ERSlists where:;
                P_j fits;
                ERS_k is closest to the Bottom-Left corner of S_i;
            if ERS_k is not empty then
                Place P_j at the Bottom-Left corner of ERS_k;
                Update ERSlists using the Difference and Elimination process;
            end
        end
    end
end
```

Algorithm 1: Heuristic placement Pseudocode

E. **Scoring**

Each chromosome will be scored as follow: Suppose that k stocks $S_1, S_2, .., S_k$ are occupied, the i-th stock S_i has dimension $L_i \times W_i$. We call $smallpiece_i$ the smallest piece that occupied S_i and $largesetERS_i$ the largest Empty Rectangular Space after cutting all the pieces. Then:

$$score = \frac{1}{TotalArea} \sum l_j w_j - \frac{K}{TotalArea^2} \sum_{i=1}^{k} smallpiece_i * largestERS_i$$

where $TotalArea = \sum_{i=1}^{k} L_i W_i$ is the total area of all the occupied stocks, $\sum l_j w_j$ is the total area of all the pieces that fit in, and K is a small constant. In our code specifically, we chose $K = 0.03$ as the paper suggested. The reason for why this model is better than the "traditional" one (in the sense

of only consider the trim waste) is explained in Section 3 of [15]. We want to minimize this score, e.g. the lower it is, the better the result.

F. **Crossover and Mutation**
To create a new generation, we first pick 20% of the chromosomes in the current generation which have the lowest score, check for Mutation and perform the Crossover process.

- The Crossover is as follow: Suppose there are two parents Chromosomes A and B are selected to create a new chromosome. Randomly pick one gene from A or B. If this selected gene has not been chosen before, put this gene to the new chromosome. If this gene has already appeared in the new chromosome, then select a gene from the other parent chromosome. If this selected gene does not appear in the crossover chromosome, then assign it to the crossover chromosome. If not, leave the position of the gene in the crossover chromosome blank to make sure the genes in the crossover chromosome will not be duplicated. This bitwise process continues until all the genes in the parent chromosomes are processed. Then the blank positions are filled with those unselected genes to form the final child chromosome.
- There is a 10% chance for mutation in each generation. When mutation happens, instead of populating the next generation by Crossover the top 20% chromosomes, we spare 20% and populate it with the 20% worst chromosomes in the current generation.

G. **Multi-Phase Genetic Algorithm (MPGA)**
This is our main algorithm. The idea behind MPGA is to improve the diversification capability of the basic GA algorithm. It implements the basic GA algorithm times (phases). The best ϕ solutions from the last generation in a phase will automatically become members in the initial population for the next phase, and the remaining members in the initial population will be generated by a GA crossover operator. This multi-phase design will not only improve the diversification capability of the basic GA algorithm, but also guide the search to good solution regions. The pseudo-code of the multi-phase GA algorithm is presented below 2:

```
Data: Problem parameters, maximum generations maxgen, number of phases m
Result: Best solution found
for multi-phase = 0 to m do
    if multi-phase = 0 then
        Initialize all populations;
    else
        Reset all parameters to initial conditions;
        parents ← best solutions from previous phase + randomly generated solutions;
    end
    for generation = 1 to maxgen do
        Evaluate the fitness values for the new population and save the best solutions;
        Perform Selection, Crossover, and Mutation to generate new populations;
    end
end
```

Algorithm 2: Multi-phase Genetic Algorithm

In our implementation, we have m = 7, there will be 20 generations in each phase, and the best 15% will be used to generate the next phase.

3.1.2 Experimental results and algorithm analysis

In this section, we will present results from our code as well as some analysis on its performance.

- Number of episodes: 100; seed = 123
- Average filled ratio: 0.1809
- Average trim loss: 0.1377
- Average time taken: 16.5259 seconds

We can see that the performance of the Genetics Algorithm is not that bad, especially when it only runs 5 phases with 20 generations each phase and has only 20 chromosomes in the population comparing to the suggested value of 30-40 times the length of a chromosome, which is at least 3000!
Next, we analyze the time complexity of the GA. First, denote n the length of a chromosome and P the size of the population. Note that the function that produces the next generation from the current one calls the following functions:

- The Heuristic placement function: This function runs in $O(n^2)$ since we iterate through each stock and each demand. Doing this for all chromosomes yields $O(Pn^2)$ time complexity.
- The Scoring function: This function is pretty straightforward. It just iterates through the chromosome and calculates the score in $O(n)$. Doing this for all chromosomes yields $O(Pn)$ time complexity.
- The Sorting function: This function uses Merge Sort to sort the chromosomes by their score to perform Crossover. Merge Sort runs in $O(P\log(P))$.
- The Crossover function: Creates the new chromosome based on two parent chromosomes. It runs in $O(n)$. However, since we have to populate the whole generation, the time complexity amounts to $O(Pn)$ where P is the size of the population.

Therefore, the time complexity for to populate a new generation is $O(Pn^2+P\log(P))$. Since the algorithm runs in m phases, having G generations for each phase, the total time complexity of the algorithm is $O(mGP(n^2+log(P)))$. When m, G and P are fixed values like in our implementation, the time complexity degenerates to $O(n^2)$.

3.1.3 Advantages and disadvantages

- Advantages:
  - Can achieve very good results within a reasonable number of phases and generations.
  - Less expensive to compute than other metaheuristics such as TABU or GRASP while still being as good.
  - The idea behind the algorithm is very natural and thus very easy to understand. Because of such simplicity, parallelization of this algorithm is also viable and is, in fact, relatively easy to implement.
- Disadvantages:
  - Being less expensive than some other algorithms, yet it does not mean that it is cheap to compute. Indeed, when inspecting the time complexity, one can see that even though the algorithm runs in $O(n^2)$, the hidden constant is abysmally large, resulting in poor runtime.
  - A common problem for many genetic algorithms is that there are many factors to keep track of, such as population size, mutation rate, solution space, etc. Therefore, poorly chosen parameters can lead to a local optimum.

3.2 Greedy algorithm

3.2.1 Pseudocode explanation

In general, a greedy algorithm tries to reach the optimal solution of a problem by picking the choices that 'look most optimal currently'. The optimality of a choice is determined by some criteria that the best solution possesses and can be checked without backtracking previous choices. In our algorithm, assuming the height of a rectangle is its shorter side and the width is its longer one, it prioritizes filling wider stocks first, also with wider pieces first. More clearly, the steps are as follows.

- Prepare the stocks and pieces by reorienting them so that the heights of all stocks and pieces are smaller than their respective width, then sort the stocks and pieces by width.
- Iterating through each stock from the widest and doing the next two steps for each of them.
- Phase 1: Iterating through each piece from the widest and putting each of them 'flat down' into the stock – as in both width sides are parallel to each other – if possible.

- Phase 2: Iterating through the pieces and trying to put them in stock again, but this time the pieces are put 'standing up' - as in their width, the sides must be perpendicular to the stock.

The pseudocode described by those steps is as follows in 3.

```
Input: Stocks, Pieces
Output: Cut Instruction

foreach rectangle R in Stocks and Pieces do
    if R.width < R.height then
        Rotate R;
    end
end
Sort(Stocks, order = widest_first);
Sort(Pieces, order = widest_first);
foreach stock S in Stocks do
    foreach piece P in Pieces do
        Iterate through points in S from left to right, low to high;
        if area of P at (x, y) is not marked then
            break;
        end
        if (x, y) is found then
            Append CutInstruction.append(Cut on stock S at (x, y) a piece P);
            Mark the fitted area of P on S and remove P from Pieces;
        end
    end
    foreach piece P in Pieces do
        Iterate through points in S from right to left, low to high;
        if area of P.rotate() at (x, y) is not marked then
            break;
        end
        if (x, y) is found then
            Append CutInstruction.append(Cut on stock S at (x, y) a piece P.rotate());
            Mark the fitted area of P.rotate() on S and remove P from Pieces;
        end
    end
end
```

Algorithm 3: Cutting Instructions for Stocks and Pieces

3.2.2 Trim loss and Filled ratio analysis

From running the implemented Python program for this algorithm and measure its results, we have

- Number of episodes: 100
- Average filled ratio = 0.1601
- Average trim loss = 0.1777

which is somewhat worse than the genetic algorithm. This is expected because unlike it, the greedy algorithm does not backtrack its choice and pick a worse one to hopefully reach a better local minima. In exchange, the greedy algorithm is faster and easier to implement.
Visually, we can categorize pieces packing in each stock into 3 levels based on trim loss:

- Good stocks: since pieces are packed bottom-up and left-to-right, and the pieces are packed flat down in phase 1, wastes often remain only at the top and right border of the stock. The right border can filled furthermore in phase 2, when packed pieces are standing up and therefore parallel to the border. Good stocks are stocks with small trim loss, and they usually utilize this advantage to lower the trim loss.

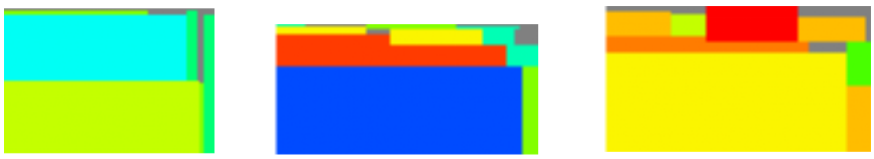


Figure 5: Good stocks

- Acceptable stocks: the algorithm does not sort pieces by height, so the next packed piece can be either taller or shorter than the current packed piece. If the next piece is high enough, one of its height and one of the width side of the current piece will form a corner, and this corner usually becomes the corner of a wasted area. The trim loss of Acceptable stocks is close to the average trim loss, and this is due to these "holes" appearing in them.

Figure 6: Acceptable stocks

- Bad stocks: Sometimes, for the last stocks to be packed, there are not enough variety of pieces left, so the waste at the top and right corners is much more. Sometimes, the total area of the remaining pieces for the last stock is very small compared to the stock, so the pieces cannot efficiently fill the stock. Bad stocks are either one of those types.

Figure 7: Bad stocks

3.2.3 Advantages, disadvantages and further discussion

The algorithm gives relatively good cutting patterns in a short time. Moreover, it is simple to follow and implemented, and therefore can be easily improved.
However, it needs to precompute and store information about the stocks and pieces to be working properly. If given partially cut stocks, the algorithm likely breaks. We can avoid the problem by searching for the best stock and piece every time, however this will increase runtime because we now have to search every time instead of sorting everything once.
In our tests, the cut stocks are mostly good stocks. However we cannot predict how many acceptable stocks and bad stocks there are. This is due to our algorithm not trying to plan ahead or backtracking to avoid having holes in the stocks or using very large stock to fill the remaining small pieces. Perhaps we can add better heuristics to the algorithm, permute the arrangement of pieces in a stock to try to use the extra wastes. However, the algorithm will be complicated.

4 Conclusion

In summary, we seek to understand a few basic concepts concerned with the cutting stock problem in general, together with discussing the cutting stock problem in this assignment in particular, in terms of mathematical models, examination, implementation and evaluation on two specific algorithms.

We have introduced shortly when and how the cutting stock problem has gained popularity in the literature. Then, a integer programming formulation of our problem and a discussion on cutting problem classification have been presented. After knowing the problem's model in this assignment and how to identify the cutting stock problem's variations, we have shown a brief summary of some exact methods and heuristics.

Additionally, we have discussed two algorithms, one of which performs better but is more intricate and needs careful consideration; while the other's implementation is based on a more straightforward approach.

References

[1] A. I. Hinxman. "The trim-loss and assortment problems: A survey". In: *European Journal of Operational Research* (1980).

[2] F. G. Ortmann. *Heuristics for offline rectangular packing problems*. 2010. URL: https://www.researchgate.net/publication/45635278_Heuristics_for_offline_rectangular_packing_problems.

[3] G. Wascher, H. Haussner, and H. Schumann. "An improved typology of cutting and packing problems". In: *European Journal of Operational Research* (2007).

[4] H. Ben Amor and J.M Valério de Carvalho. "Cutting Stock Problems". In: *Column Generation*. Springer New York, NY, 2005, p. 3.

[5] H. Dyckhoff. "A topology of cutting and packing problems". In: *European Journal of Operational Research* (1990).

[6] J. F. Goncalves. "A hybrid genetic algorithm-heuristic for a two-dimensional orthogonal packing problem". In: *European Journal of Operational Research* 183.3 (2007), pp. 1212–1229. DOI: 10.1016/j.ejor.2005.11.062. URL: https://doi.org/10.1016/j.ejor.2005.11.062.

[7] K. Eisemann. "The trim problem". In: *Management Science* (1957).

[8] K. K. Lai and Jimmy W. M. Chan. "Developing a simulated annealing algorithm for the cutting stock problem". In: *Computers & Industrial Engineering* 32.1 (1997), pp. 115–127. DOI: 10.1016/S0360-8352(96)00205-7. URL: https://doi.org/10.1016/S0360-8352(96)00205-7.

[9] L. V. Kantorovich. "Mathematical methods for organising planning production". In: *Management Science* (1960).

[10] N. Christofides and C. Whitlock. "An Algorithm for Two-Dimensional Cutting Problems". In: *Operations Research* (1977).

[11] P. C. Gilmore and R. E. Gomory. "A linear programming approach to the cutting-stock problem". In: *Operations Research* (1961).

[12] R. Cucuzza et al. "Sustainable and cost-effective optimal design of steel structures by minimizing cutting trim losses". In: *Automation in Construction Automation in Construction* (2024), pp. 4–5.

[13] R. Morabito and V. Garcia. "The cutting stock problem in a hardboard industry: A case study". In: *Computers & Operations Research* (1998).

[14] T. Stuzle. *Local search algorithms for combinatorial problems - analysis, improvements, and new applications*. DISKI—Dissertationen zur Kunstliken Intelligenz. Sankt Augustin, Germany: Infix, 1999.

[15] T. Y. Yu et al. "Applying an Enhanced Heuristic Algorithm to a Constrained Two-Dimensional Cutting Stock Problem". In: *Applied Mathematics & Information Sciences* (2014). DOI: 10.12785/amis/091L23. URL: http://dx.doi.org/10.12785/amis/091L23.

[16] U. Ayasandir. "Two dimensional cutting stock problem with multiple stock sizes". PhD thesis. Middle East Technical University, 2019.

[17] Y. Lirov. "Knowledge based approach to the cutting stock problem". In: *Mathematical and Computer Modelling* (1992).